

Programación agéntica en el día a día

Cinco piezas para tratar a la IA como un compañero de equipo, no como un autocompletado

Un agente sin contexto y sin proceso es un becario con prisa. Con contexto y con proceso es un compañero de equipo.

Resumen de la sesión · 8 páginas · 10 min de lectura

Template de referencia: `ddd-fullstack-starter`

Las 5 piezas

QUÉ LE DOY AL AGENTE (sustrato)

1. **UNA SOLA FUENTE DE VERDAD**
 - `.agents/` canónico + symlinks + `AGENTS.md`
2. **GUIDELINES COMO CONTEXTO**
 - Mis convenciones cargadas en cada conversación

CÓMO LO USO (maquinaria)

3. **AGENTES, SUBAGENTES Y SKILLS**
 - Un equipo con roles, no un asistente
4. **OPENSPEC**
 - Especificar antes de implementar
5. **OPENCODE POR FASE**
 - Cada fase, el cerebro y los permisos justos

Cómo leer este handout: una página por pieza. Cada una con el problema que resuelve, cómo se monta, un snippet real y la frase que recordar.

1 · Una sola fuente de verdad

El problema

Cada herramienta de IA quiere su carpeta: `.claude/`, `.cursor/`, `.github/copilot/`, `.opencode/...` Duplicar contenido en cada una garantiza que en dos semanas estén desincronizadas.

Cómo se monta

- **Una carpeta canónica:** `.agents/` CON `agents/` y `skills/`.
- **Las demás son symlinks** a esa carpeta única.
- `AGENTS.md` **en la raíz** como punto de entrada universal (Cursor y Copilot lo leen directamente).

```

.agents/                <- fuente de verdad
  agents/              <- roles (code-reviewer, etc.)
  skills/              <- prompts cargables

.claude/agents  -> ../.agents/agents    <- symlink
.claude/skills  -> ../.agents/skills    <- symlink

AGENTS.md          <- entrada universal

```

Regla operativa

Edita siempre en `.agents/`. Nunca a través de los symlinks. Referencia `.agents/...` en config y docs. Nunca `.claude/...`

La idea

Edito en un sitio. Lo leen todas las herramientas. Cero duplicación.

2 · Guidelines como contexto

El problema

Si tus convenciones viven en tu cabeza, el agente las ignora. Si viven en un wiki, nadie las lee. Si viven en el repo pero no se cargan, el agente improvisa.

Cómo se monta

- **Convenciones como skills** en `.agents/skills/guidelines/` (naming, arquitectura, testing, TDD, frontend, git).
- **Formato corto:** 30-50 líneas por skill, checklist accionable, sin paja.
- **Carga automática** vía `instructions[]` en `opencode.json`: se inyectan en *cada* conversación.

```

// opencode.json
"instructions": [
  "AGENTS.md",
  ".agents/skills/guidelines/design-principles/SKILL.md",
  ".agents/skills/guidelines/hexagonal-architecture/SKILL.md",
  ".agents/skills/guidelines/tdd-practices/SKILL.md",
  ".agents/skills/guidelines/testing-standards/SKILL.md",
  ".agents/skills/guidelines/frontend-patterns/SKILL.md",
  ".agents/skills/guidelines/git-strategy/SKILL.md"
]

```

Cabecera estándar de una skill

```
---
name: design-principles
description: Design, naming, and error-handling rules - load when
            writing or reviewing TypeScript code in packages/.
---
```

El description le dice al agente **cuándo** cargar la skill.

La idea

El agente no adivina mi estilo. Lo lee en cada conversación.

3 · Agentes, subagentes y skills

Modelo mental

Pieza	Es	Vive en
Skill	Un prompt cargable (cómo hacer algo)	.agents/skills/
Agente	Un rol: modelo + tools + skill cargada	.agents/agents/
Subagente	Un agente al que otro agente puede invocar	mismo sitio, mode: subagent

Las **skills** son el QUÉ hacer. Los **agentes** son el QUIÉN lo hace.

Roles típicos del equipo

```
code-reviewer · architecture-reviewer · tests-reviewer · frontend-reviewer · spec-writer
· spec-reviewer · project-validator
```

Cada rol con un fichero .md que define: **descripción, tools permitidas, constraints (lo que NO puede hacer), scope, qué revisar, formato de output.**

La regla que más cambia el juego

Mandatory Review Gate

After modifying production code, you **MUST** run code-review, tests-review, and architecture-review **IN PARALLEL** before reporting the task as complete. Non-negotiable.

Cuando el agente principal termina, **dispara los tres reviewers en paralelo**. Tú recibes código ya revisado por tres roles distintos.

La idea

Tengo un equipo, no un asistente. Cada rol hace una cosa y la hace bien. El humano es el último reviewer, no el único.

4 · OpenSpec — especificar antes de implementar

El problema

El agente que improvisa **deriva en 20 minutos**. Empieza bien, se va por las ramas, y acabas con código que no encaja con tu arquitectura.

La solución

Spec Driven Development: la spec es el **contrato** entre tú y el agente. Cuatro fases, una carpeta por cambio:

```
explore  ->  pensar, investigar el código
           NO escribe nada

plan      ->  generar proposal.md, design.md,
           specs/ (Given/When/Then), tasks.md
           escribe SOLO dentro de openspec/

build     ->  ejecutar las tareas del plan
           aquí sí toca el código de verdad

archive   ->  cerrar el cambio, mover a archive/,
           actualizar specs canónicas
```

Anatomía de un cambio

```
openspec/changes/<change-id>/
proposal.md    <- qué y por qué (con non-goals)
design.md       <- cómo (capas, ports, adapters)
specs/         <- escenarios Given/When/Then
tasks.md       <- pasos ordenados inside-out,
                con TDD donde haya behavior
```

Por qué funciona

- El **plan se valida antes** de escribir código. Iterar sobre un markdown es barato; iterar sobre código es caro.
- Las **tasks marcadas** ([x]) dan trazabilidad real de qué se hizo.
- Al **archivar** el cambio queda como historia, no como ruido.

La idea

No le pido código. Le pido un plan. Después ejecuto el plan.

5 · OpenCode — el cerebro adecuado por fase

El problema

Usar el mismo modelo, la misma temperatura y las mismas tools para *todo* es caro y peligroso. Opus para mover archivos es desperdiciar dinero. Permitir `write` en una fase de exploración es invitar al desastre.

La solución: modos

Cada fase de OpenSpec se materializa como un **modo** con su modelo, temperatura y permisos de tools.

```
// opencode.json (extracto)
"explore": {
  "model": "claude-opus-4.7",
  "temperature": 0.7,
  "reasoningEffort": "high",
  "tools": { "write": false, "edit": false, "bash": false }
},
"plan": {
  "model": "claude-opus-4.7",
  "temperature": 0.2,
  "reasoningEffort": "high",
  "tools": { "write": true, "edit": true } // solo en openspec/
},
"build": {
  "model": "claude-sonnet-4.5",
  "temperature": 0.2,
  "reasoningEffort": "medium",
  "tools": { "write": true, "edit": true, "bash": true }
},
"archive": {
  "model": "claude-sonnet-4.5",
  "temperature": 0.1,
  "reasoningEffort": "low"
}
```

Las tres ideas detrás

1. **Cada fase usa el cerebro que necesita.** Opus para pensar y planear, Sonnet para ejecutar y archivar.

2. **Las tools están restringidas por fase.** En `explore`, el agente literalmente no puede escribir. Aunque tú se lo pidas.
3. **La temperatura también cambia.** 0.7 para explorar (creativo), 0.1 para archivar (mecánico).

La idea

Cada fase usa el cerebro que necesita. Ni más caro, ni más permissivo.

Por dónde empezar el lunes

No intentes montar las 5 piezas a la vez. Sigue este orden, una semana cada paso:

Paso 1 · Escribid vuestro `AGENTS.md` (**1 tarde**)

Un solo archivo en la raíz del repo. Stack, arquitectura, convenciones clave, comandos. Si Cursor, Copilot o Claude entra mañana al repo, **esto es lo primero que tiene que leer**. Solo con esto, los resultados mejoran notablemente.

Paso 2 · Extraed vuestras guidelines a archivos (**1-2 días**)

Lo que repetís en cada code review se convierte en una skill: `naming`, `error-handling`, `testing`, lo que sea vuestro. Una skill = una checklist corta y accionable. Cargadlas siempre vía la config de vuestra herramienta.

Paso 3 · Definid UN subagente reviewer (**medio día**)

Empezad por `code-reviewer`. Un `.md` con: qué revisa, qué NO toca, formato de output. Invocadlo manualmente al principio. Cuando confiéis, automatizadlo como gate obligatorio antes de cerrar tareas.

Paso 4 (opcional, cuando los anteriores estén asentados)

Spec Driven Development con OpenSpec, y modos por fase en vuestra herramienta de IA. Solo tiene sentido cuando los pasos 1-3 son hábito.

Para profundizar

- **Template de referencia:** `ddd-fullstack-starter` (*repo público*)
- **Documentación de convenciones:** `docs/conventions/`
- **OpenSpec:** github.com/Fission-AI/OpenSpec
- **OpenCode:** `opencode.ai`

Recordad la idea

Un agente sin contexto y sin proceso es un becario con prisa. Con contexto y con proceso es un compañero de equipo.